



Παράλληλος και Κατανεμημένος Υπολογισμός I

Παράλληλος υπολογισμός (συν.3)

Κ.Γ. Μαργαρίτης
Πανεπιστήμιο Μακεδονίας
Τμήμα Εφαρμοσμένης Πληροφορικής
2025



Διαίρει και βασίλευε

Διαίρει και βασίλευε 1

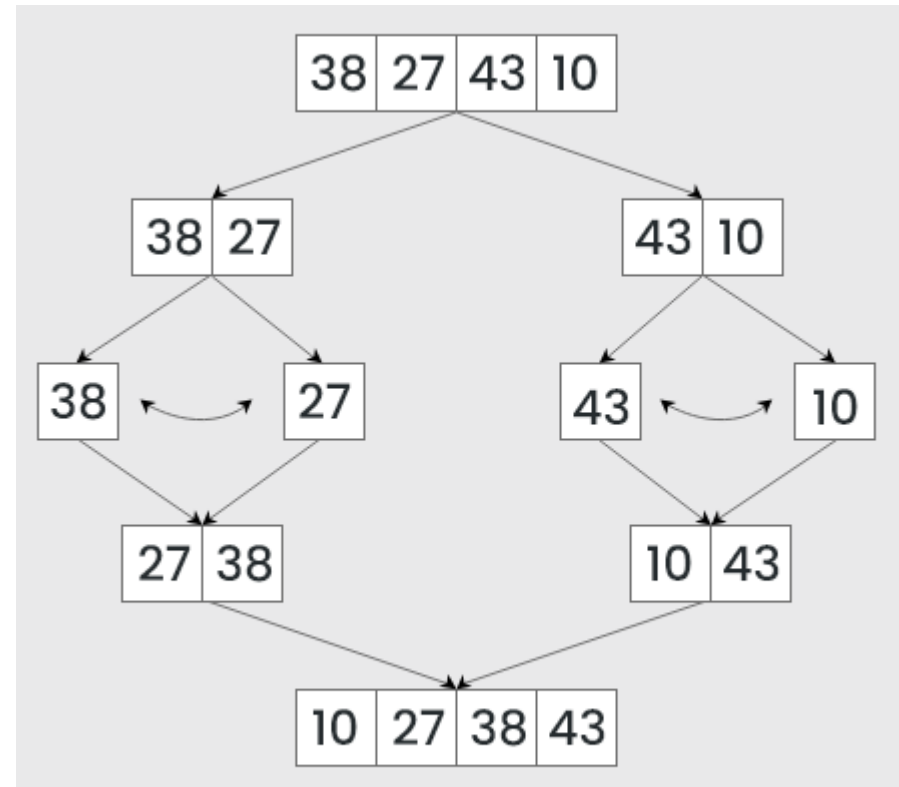
- Τα δεδομένα υποδιαιρούνται αναδρομικά σε blocks υποδιπλάσιου(συνήθως) μεγέθους και ανατίθενται σε αναδρομικά δημιουργούμενες εργασίες.
- Συνήθως υπάρχει και αντίστροφη διαδικασία συλλογής του αποτελέσματος, δηλαδή αναγωγή.
- Χρήσιμη όταν ο αλγόριθμος έχει αναδρομική μορφή γιατί επιτρέπει άμεση απεικόνιση του αλγορίθμου στο παραλληλισμό.
- Χρησιμοποιείται και γενικότερα γιατί επιτρέπει εύκολη ρύθμιση του μεγέθους των τελικών blocks καθώς και του πλήθους των δημιουργούμενων εργασιών.

Διαίρει και βασίλευε: Merge Sort

x[n];

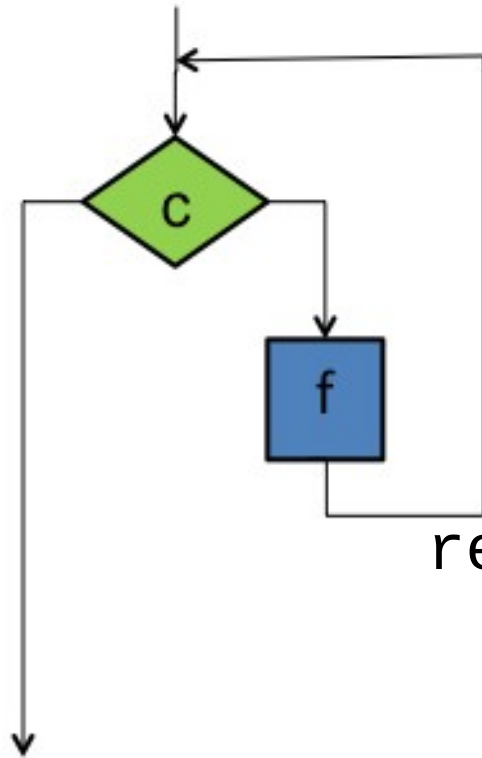
main: merge_sort(0, n);

```
merge_sort(from, to) {  
    size = to - from;  
    if size > 1 {  
        mid = from + size / 2;  
        merge_sort(from, mid);  
        merge_sort(mid, to);  
        merge(from, mid, to);  
    }  
    merge(from, mid, to) {...  
}
```



Διαίρει και βασίλευε:

Αναδρομική απαριθμήσιμη επανάληψη



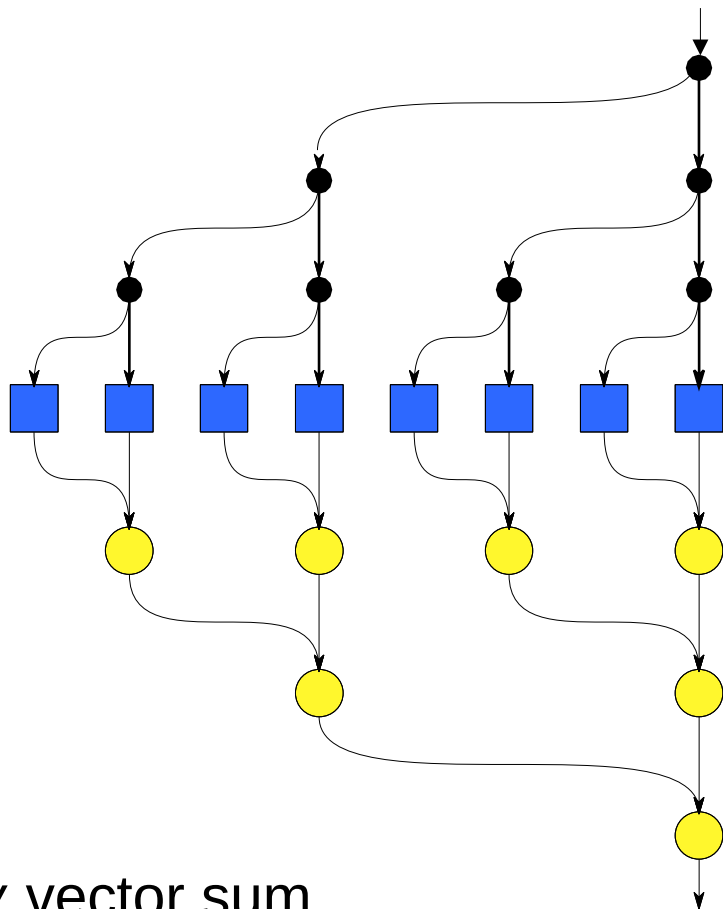
```
for (i = 0; i < n; ++i)
    y = f(x, i);
```

↖ Σώμα της επανάληψης

Για ανεξάρτητες επαναλήψεις:

```
recursive_for (from, to) {
    size = to - from;
    if (size == 1)
        y[from] = f(x, from);
    else {
        mid = from + size / 2;
        recursive_for(from, mid);
        recursive_for(mid, to);
    }
}
```

Διαίρει και βασίλευε: Αναδρομική απεικόνιση και αναγωγή



Sum (0,7) = Sum(0,3)+Sum(4,7)
Sum (0,3) = Sum(0,1)+Sum(2,3)
Sum (4,7) = Sum(4,5)+Sum(6,7)
..

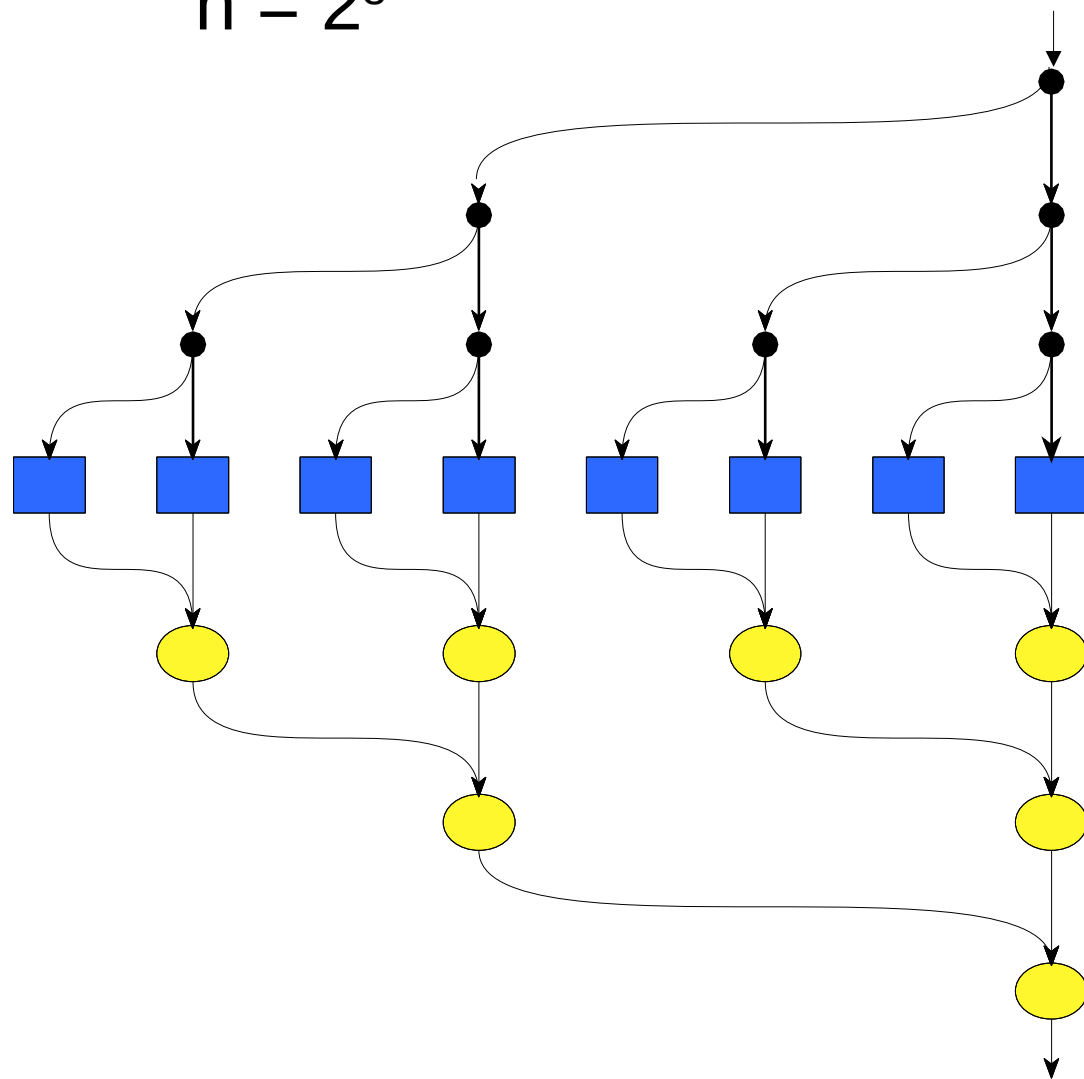
```
rec_mr:recursive map reduce
rec_mr (from,to) {
    size = to-from;
    if (size == 1)
        out = map(a[from]);
    else {
        mid = from+size/2;
        outL = rec_mr(from, mid);
        outR = rec_mr(mid, to);
        out = reduce(outL, ourR);
    }
}
```

Διαίρει και βασίλευε 1

- Για αρχικό πρόβλημα μεγέθους $n = 2^k$, και με δεδομένα $x[n]$
- Κάθε αναδρομική κλήση υποδιπλασιάζει το block, δηλαδή τα μεγέθη των blocks είναι: $n, n/2, n/4 \dots 1$
- Κάθε αναδρομική κλήση δημιουργεί 2 εργασίες. Ξεκινούμε από μια εργασία. Δηλαδή έχουμε: $1, 2, 4, 8, \dots, n$ εργασίες.
- Το μέγιστο βάθος της αναδρομής είναι $\log n = k$ και συνολικά δημιουργούνται: $1+2+4+\dots+2^k = 2^{k+1} - 1$ εργασίες.

Διαίρει και βασίλευε 2

$$n = 2^3$$



Απεικόνιση :
 $\log n = 3$ βήματα

Βασικός υπολογισμός:
 $n/n = 1$ βήμα σε κάθε
τελική εργασία

Αναγωγή:
 $\log n = 3$ βήματα

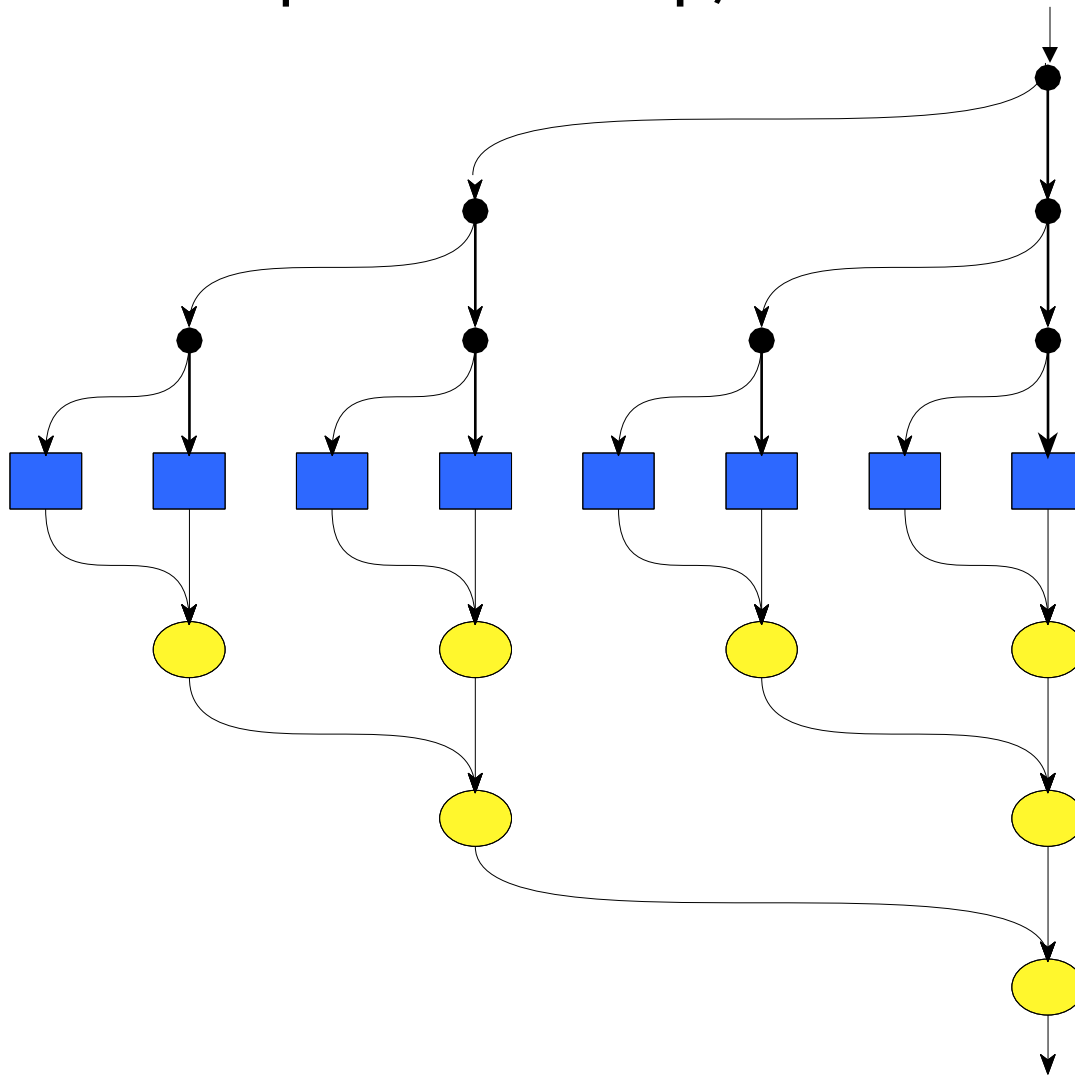
Διαίρει και βασίλευε 3

- Μπορούμε να ρυθμίσουμε το πλήθος των εργασιών ρυθμίζοντας το μέγεθος του τελικού block, δηλαδή το κατώφλι αποκοπής της αναδρομής.
- Πχ, αν σταματήσουμε στο μέγεθος τελικού block στο $2^2=4$ τότε οι δημιουργούμενες εργασίες είναι:
1, 2, 4, 8, ..., $n/4$ εργασίες.
- Το μέγιστο βάθος της αναδρομής είναι $\log n = k-2$ και συνολικά δημιουργούνται $1+2+4+..+2^k = 2^{k-1} - 1$ εργασίες.

Διαίρει και βασίλευε 4

$$n = 2^8$$

Κατώφλι αποκοπής: 2^5 .



Απεικόνιση :
 $\log (8-5) = 3$ βήματα

Βασικός υπολογισμός:
 2^5 βήματα σε κάθε μια από
τις $2^{(8-5)} = 2^3$ τελικές εργασίες

Αναγωγή:
 $\log (8-5) = 3$ βήματα

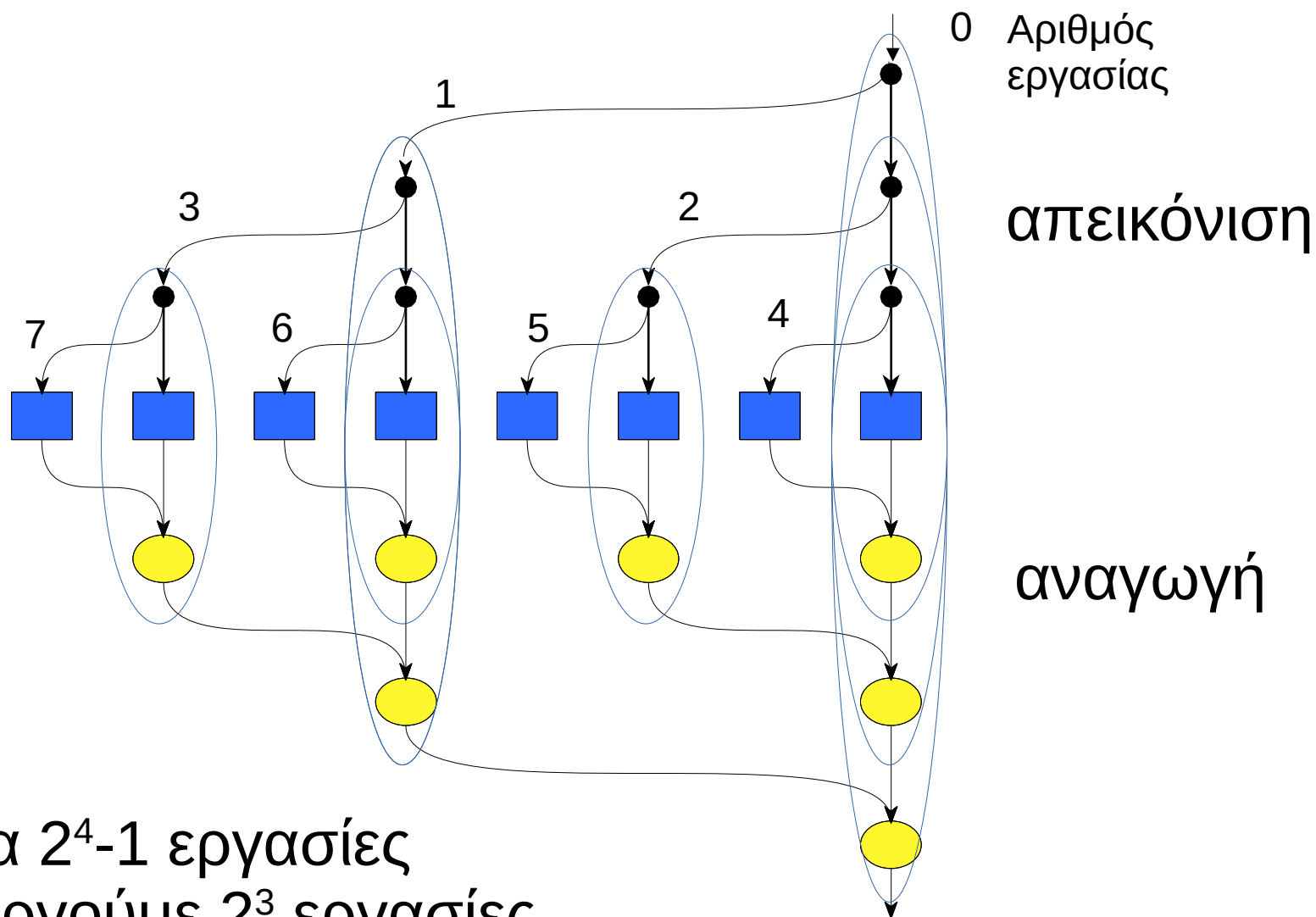
Διαίρει και βασίλευε 5

- Μπορούμε να υποδιπλασιάσουμε τον αριθμό των εργασιών αν σε κάθε βήμα η κάθε εργασία εκτελεί τον υπολογισμό του ενός κλάδου και δημιουργεί μόνο μία εργασία (δες σχήμα στην επόμενη διαφάνεια).
- Επίσης οι εργασίες ανατίθενται σε συγκεκριμένο αριθμό νημάτων, συνήθως P (όσοι οι επεξεργαστές) και δρομολογούνται από το σύστημα χρόνου εκτέλεσης, σαν υβριδικά (εικονικά) νήματα.

Πχ C/C++ CilkPlus, OpenMP Tasks, Go goroutines, Java ForkJoin Executor

- Δρομολόγηση 'κλοπής εργασιών' (Work Stealing).

Διαίρει και βασίλευε 6



Αντί για 2^4-1 εργασίες
δημιουργούμε 2^3 εργασίες

Διαίρει και βασίλευε 7

Shared x[n]

Main: y = task(0, n)

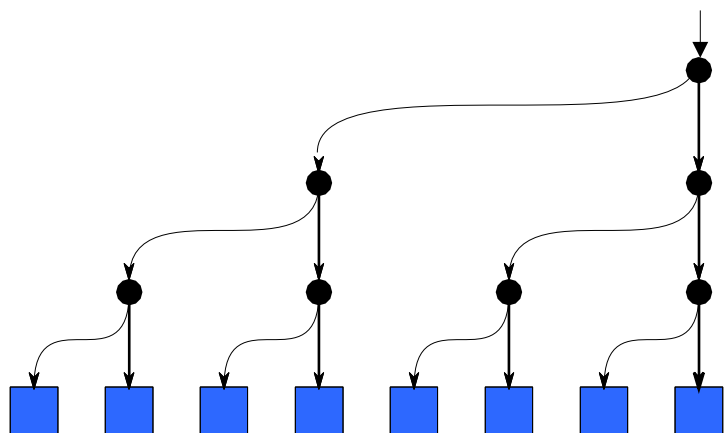
```
result task (from, to) {  
    size = to-from  
    if (size < recursion_limit)  
        return final (from, to)  
    else {  
        mid = from+size/2  
        left = fork task (from,mid)  
        right = task (mid, to)  
        return combine(left, mid, right)  
    }  
}
```

Διαίρει και βασίλευε 8

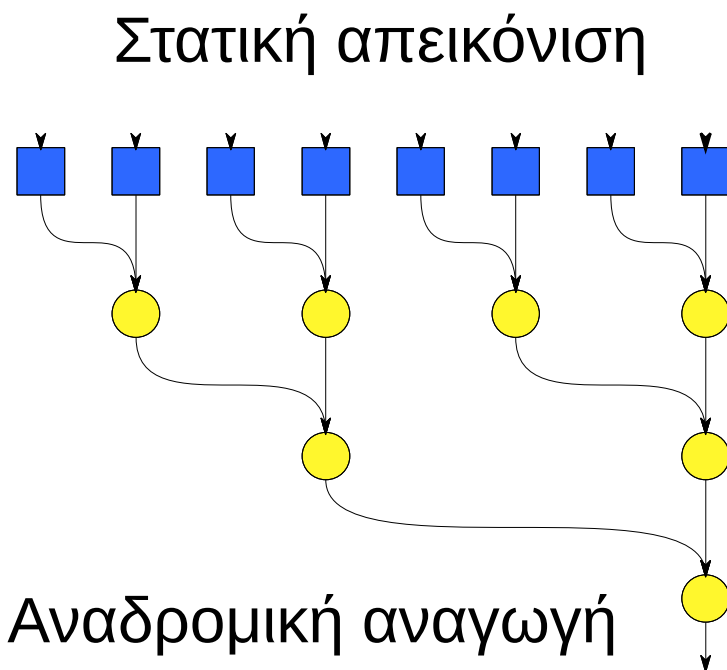
- Το `recursion_limit` καθορίζει το κατώφλι αποκοπής, δηλαδή το μέγεθος του τελικού block και αντίστοιχα τον αριθμό των δημιουργούμενων εργασιών.
- Η συνάρτηση `final()` εκτελεί ακολουθιακό υπολογισμό στο τελικό block και επιστρέφει το αποτέλεσμα.
- Το `fork` σημαίνει παράλληλη αναδρομική εκτέλεση, το απλό `task` απλή αναδρομική κλήση. Και οι δύο κλήσεις συγχρονίζονται (`join`) στο `combine()`
- Η συνάρτηση `combine()` συνδυάζει τα δύο αποτελέσματα `left`, `right`. Μπορεί να είναι κάτι απλό, όπως πχ μια πρόσθεση ή κάτι πιο σύνθετο όπως πχ ένα `merge()`.

Διαίρει και βασίλευε 9

- Μπορούμε να έχουμε αναδρομική απεικόνιση, χωρίς αναγωγή αλλά και αναδρομική αναγωγή με αρχική απεικόνιση πχ αρχική με στατική κατανομή.



Αναδρομική απεικόνιση



Αναδρομική αναγωγή

Διαίρει και βασίλευε: παράδειγμα vectorSum 1

Shared x[n] // $\pi \chi \ 2^{20} \sim 1000000$

Main: sum = task(0, n)

```
int task (from, to) {  
    size = to-from  
    if (size <  $2^{16}$ ) ~ 64000  
        return final (from, to)  
    else {  
        mid = from+size/2;  
        left = fork task (from,mid)  
        right = task (mid, to)  
        return left+right  
    }  
}
```

Διαίρει και βασίλευε: παράδειγμα vectorSum 2

```
int final (from, to) {  
    local_sum = 0  
    for (i=from, i<to, i++)  
        sum = sum + x[i]  
    return sum  
}
```

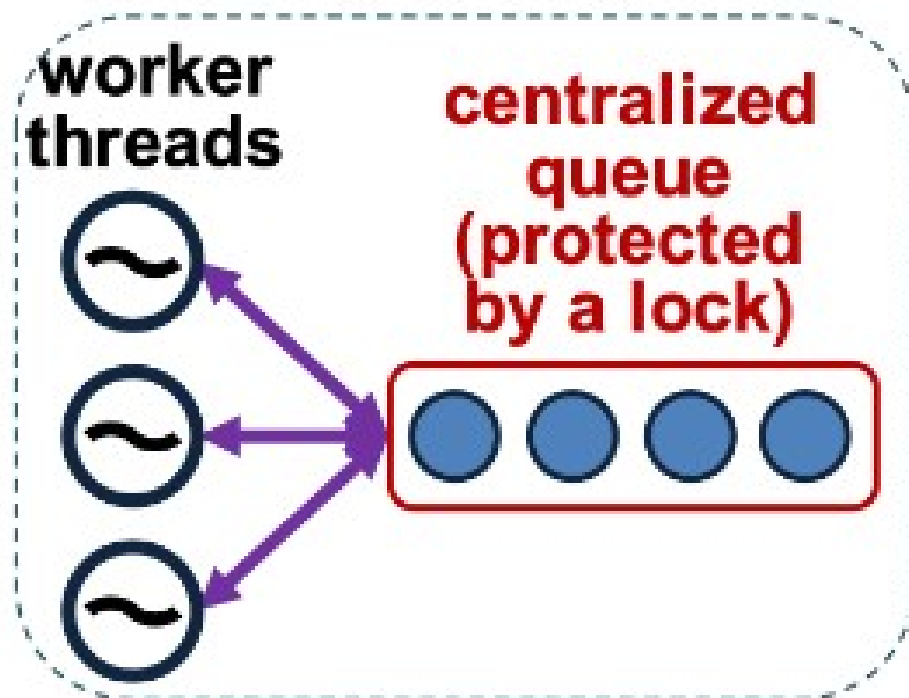
Διαίρει και βασίλευε: παράδειγμα vectorSum 3

- Δημιουργούνται $1+2+4+8+16$ εργασίες, γιατί $2^{(20-16)} = 2^4$. Άρα $2^5 - 1 = 31$ εργασίες.
- Με την τεχνική του υποδιπλασιασμού των εργασιών μπορούμε να φθάσουμε στις $2^4 = 16$ εργασίες. Έστω ότι διαθέτουμε $P = 16$ επεξεργαστές.
- Βήματα:
 2^4 βήματα υπολογισμού του final +
 $\log_{16}(4)$ βήματα αναγωγής+
 $\log_{16}(4)$ βήματα απεικόνισης (δημιουργίας εργασιών)
- Γενικά
N/P βήματα υπολογισμού του final $2^{20} / 2^4 = 2^{16}$
 $2 * \log P$ βήματα αναγωγής+απεικόνισης $2 * \log_{16} = 2 * 4$

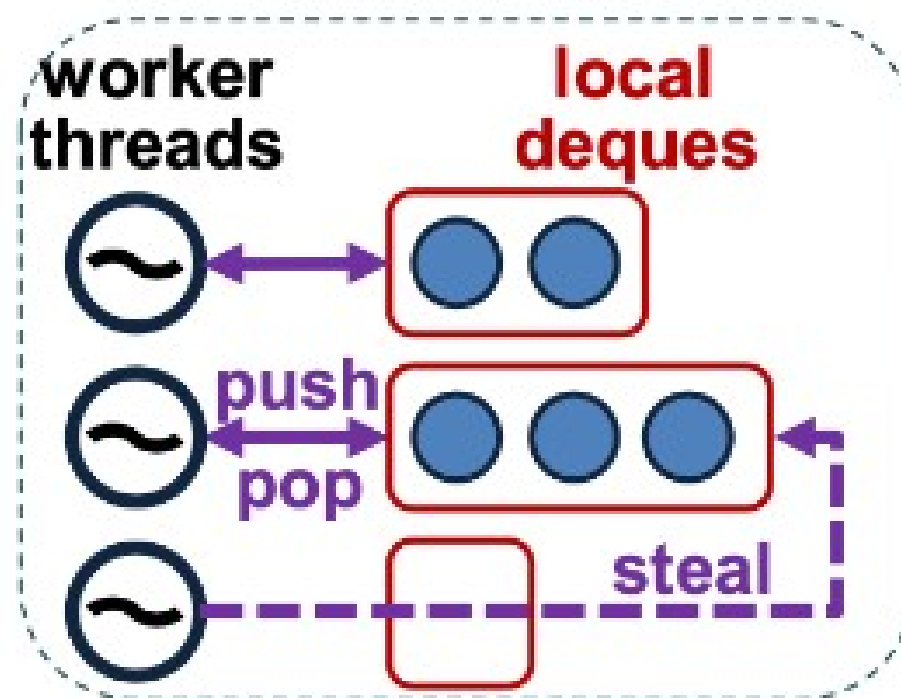
Κλοπή εργασίας (Work stealing) 1

- Αποδοτική εφαρμογή του διαίρει και βασίλευε.
- Αντί να δημιουργούμε νήματα και να αναθέτουμε χειρονακτικά την εκτέλεση των εργασιών στα νήματα, δημιουργούμε εργασίες (tasks) και αφήνουμε το σύστημα χρόνου εκτέλεσης να τις αναθέσει σε μια δεξαμενή από διαθέσιμα νήματα, συνήθως όσα και οι διαθέσιμοι επεξεργαστές.
- Αρχικά το πρώτο task ανατίθεται στο πρώτο νήμα κοκ. Όταν κάποια νήματα δεν έχουν εργασίες τότε “κλέβουν” εργασίες από τις τοπικές ουρές (στοίβες κλήσεων) των προηγούμενων νημάτων. Αυτόματη εξισορρόπηση φόρτου.

Κλοπή εργασίας (Work stealing) 2



Δυναμική κατανομή



Κλοπή εργασίας

Αρχική υλοποίηση: Cilk Plus (C/C++).

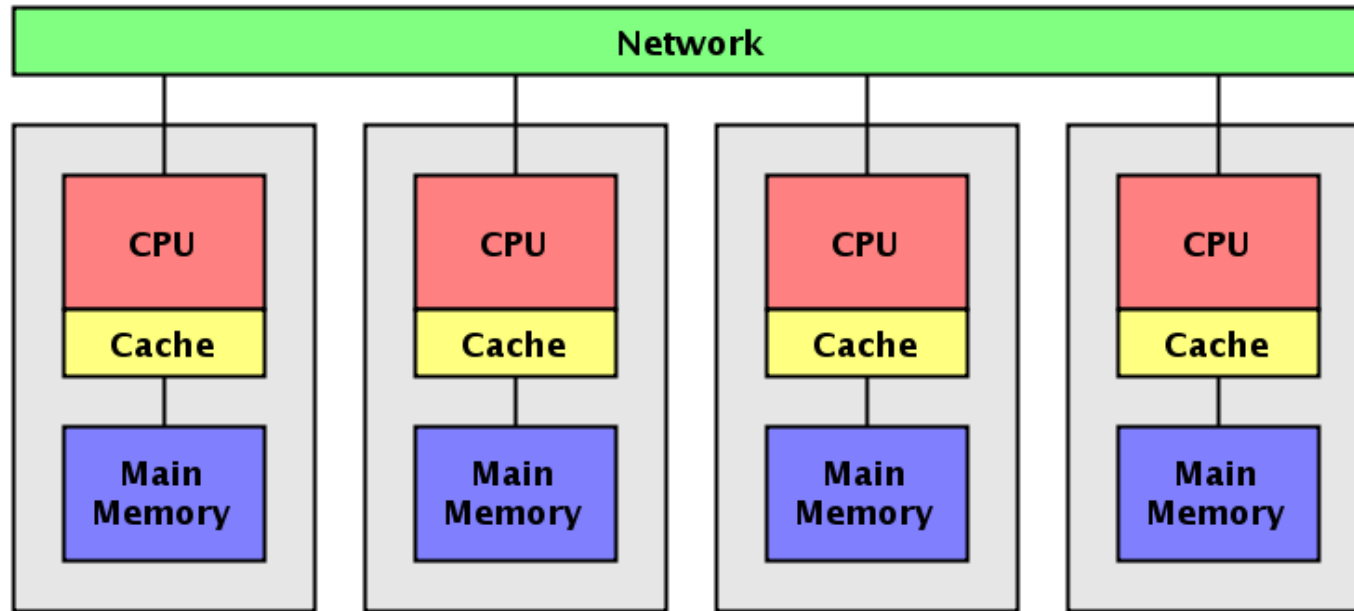


Παράλληλος και Κατανεμημένος Υπολογισμός I

Κατανεμημένη Μνήμη Μεταβίβαση Μηνυμάτων

Κ.Γ. Μαργαρίτης
Πανεπιστήμιο Μακεδονίας
Τμήμα Εφαρμοσμένης Πληροφορικής
2024

Μοντέλο υπολογιστικού συστήματος κατανεμημένης μνήμης



Ένα σύνολο από αυτόνομους υπολογιστές που επικοινωνούν μέσω δικτυακής υποδομής.

Ονομάζεται και υπολογιστικό σύστημα κατανεμημένης μνήμης σε αντιδιαστολή με τα υπολογιστικά συστήματα μοιραζόμενης μνήμης. Χρησιμοποιείται στο κατανεμημένο και στο παράλληλο υπολογισμό.

Τύποι κατανεμημένου υπολογισμού 1

Ο κατανεμημένος υπολογισμός είναι.. παντού:

- Εφαρμογές Διαδικτύου: mail, ftp, telnet, ssh ...
- Ιστός, εφαρμογές Ιστού, υπηρεσίες ιστού
- Υπολογιστικό Νέφος
- Κινητός υπολογισμός, υπηρεσίες τοποθεσίας
- Αφανής, διάχυτος υπολογισμός: διαδίκτυο των πραγμάτων, δίκτυα αισθητήρων, wearables..
- Εντατικός υπολογισμός: δεδομένα μεγάλου όγκου, συστοιχίες υψηλών επιδόσεων

Τύποι κατανεμημένου υπολογισμού 2

- Συζητούμε τα κοινά χαρακτηριστικά των διαφόρων τύπων κατανεμημένου υπολογισμού
- Αναγκαστικά κάνουμε πολλές αφαιρέσεις (abstractions) για να κρατήσουμε τα ουσιώδη

Έμφαση στα μοντέλα:

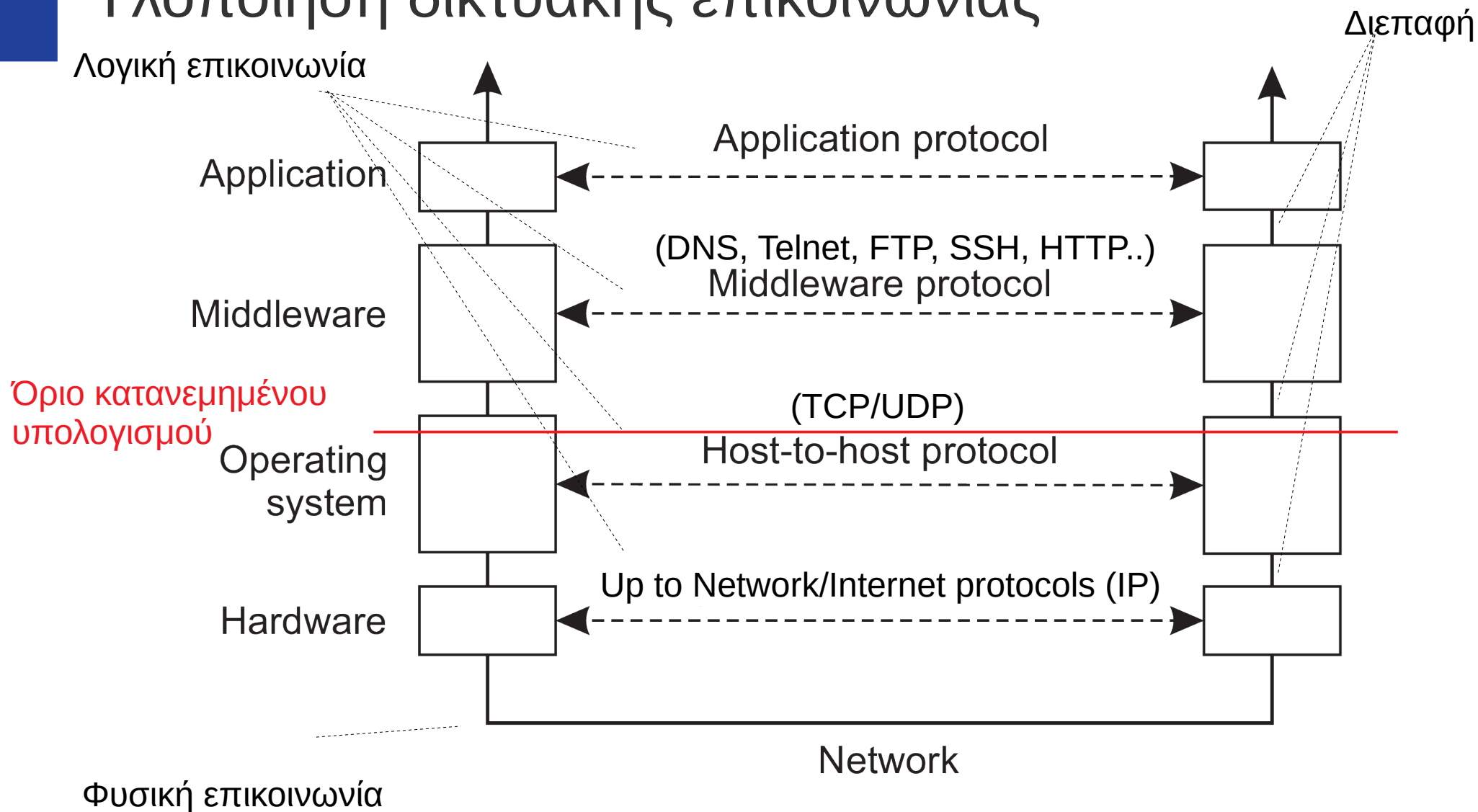
- Πελάτη Διακομιστή (Client Server)
- Συντονιστή Εργαζομένου (Master Worker)

Δύο βασικοί τύποι επικοινωνίας:

- Μηνύματα
- Απομακρυσμένη Κλήση

Υλοποίηση δικτυακής επικοινωνίας

Λογική επικοινωνία



Φυσική επικοινωνία

Μοντέλο δικτυακής επικοινωνίας για τον κατανεμημένο υπολογισμό.

Επικοινωνία host-to-host

- Στον κατανεμημένο υπολογισμό η επικοινωνία host-to-host (αντίστοιχη με επίπεδο μεταφοράς το μοντέλου OSI) αποτελεί το χαμηλότερο επίπεδο της δικτυακής επικοινωνίας το οποίο είναι προγραμματιστικά προσπελάσιμο.
- Η επικοινωνία εκτελείται από το λειτουργικό σύστημα και βασικό ενδιάμεσο λογισμικό, που υλοποιεί τα βασικά πρωτόκολλα διαδικτυακής επικοινωνίας (TCP/IP).
- Τυπικά πρωτόκολλα host-to-host επικοινωνίας: TCP και UDP.

Μεταβίβαση μηνυμάτων, πρωτόκολλα

- Η επικοινωνία είναι οργανωμένη ως μεταβίβαση μηνυμάτων (message passing).
- Μήνυμα: ένα δομημένο σύνολο από bytes. Ο αποστολέας και ο παραλήπτης μπορούν να αναγνωρίσουν τη δομή και να επεξεργαστούν το περιεχόμενο των μηνυμάτων.
- Πρωτόκολλο: σύνολο κανόνων που διέπουν την επικοινωνία. Τύποι δεδομένων, τύποι μηνυμάτων, ακολουθία μηνυμάτων, αποτελέσματα εκτέλεσης, πιθανές καταστάσεις. Συνήθως περιγράφεται ως πεπερασμένο αυτόματο.

Επίπεδα πρωτοκόλλων, διεπαφές

- Το λογισμικό επικοινωνίας (και η επικοινωνία) οργανώνεται σε επίπεδα. Έτσι έχουμε μια στοίβα πρωτοκόλλων.
- Το πρωτόκολλο κάθε επιπέδου επιλύει διαφορετικά προβλήματα και διαθέτει καλά ορισμένη διεπαφή προς τα γειτονικά του επίπεδα.
- Στον κατακεκολλημένο υπολογισμό ενδιαφερόμαστε μόνο:
 - (α) για το ανώτερο επίπεδο, δηλαδή για το πρωτόκολλο της εφαρμογής, και
 - (β) η διεπαφή με το αμέσως χαμηλότερο επίπεδο (πχ διεπαφή προγραμματισμού Υποδοχών).

Ενδιάμεσο λογισμικό (middleware)

- Επιπλέον λογισμικό που εκτελείται στους υπολογιστές ενός κατατανεμημένου υπολογιστικού συστήματος και διευκολύνει την επικοινωνία των διεργασιών (βλ. Κατατανεμημένα Συστήματα).

Παραδείγματα:

- Ενδιάμεσο λογισμικό για επικοινωνία εφαρμογών Ιστού (HTTP).
- Ενδιάμεσο λογισμικό για την ονομασία των υπολογιστών στο Διαδίκτυο (DNS).
- Ενδιάμεσο λογισμικό για την απομακρυσμένη πρόσβαση σε υπολογιστή (SSH, SFTP..)

Δικτυακή επικοινωνία: επικοινωνία διεργασιών 1

- Στον κατακευκαμένο υπολογισμό η δικτυακή επικοινωνία ισοδυναμεί με την επικοινωνία διεργασιών: μια διεργασία A που εκτελείται σε ένα κόμβο X επικοινωνεί με μια διεργασία B που εκτελείται σε ένα κόμβο Y.
- Η επικοινωνία είναι χαρακτηριστικό της διεργασίας, όπως πχ τα ανοικτά αρχεία μιας διεργασίας.
- Σε κάθε υπολογιστή (κόμβο) μπορεί να εκτελούνται ταυτόχρονα πολλές διεργασίες.

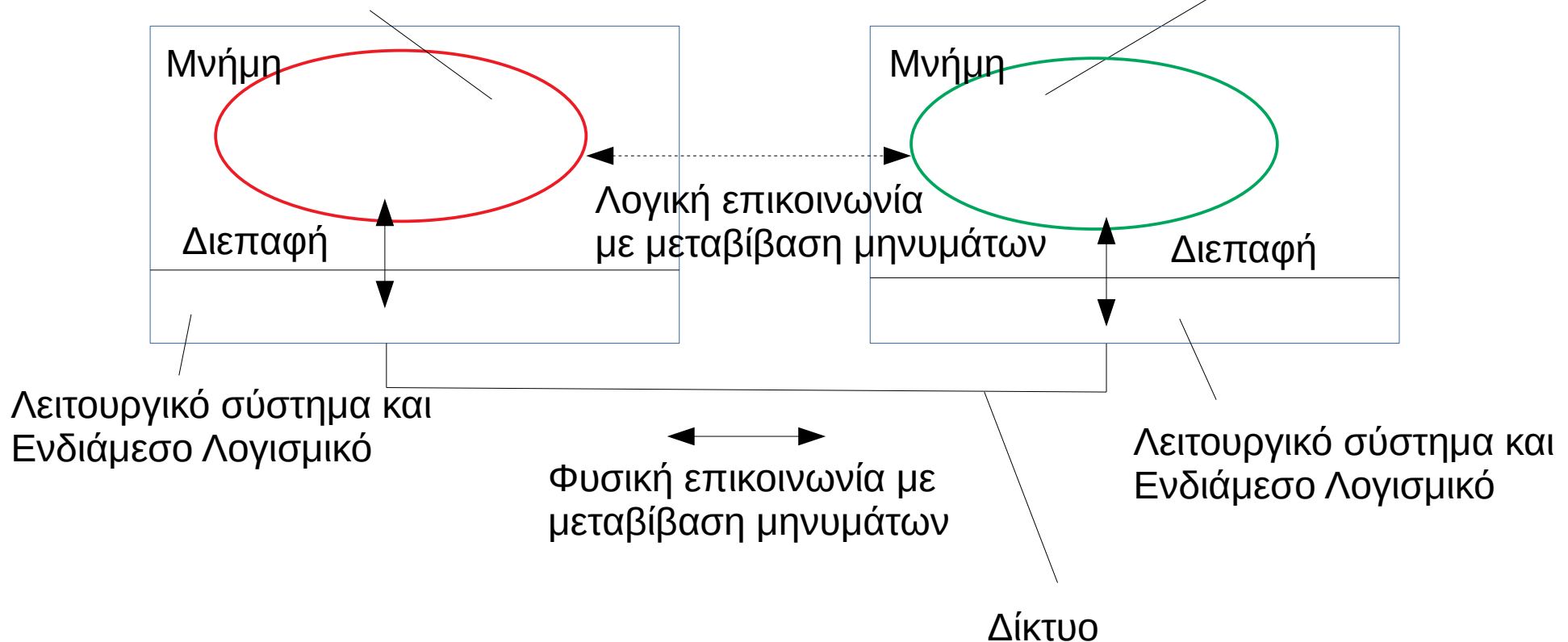
Δικτυακή επικοινωνία: επικοινωνία διεργασιών 2

- Μια διεργασία μπορεί να επικοινωνεί ταυτόχρονα με πολλές διεργασίες, δηλαδή η διεργασία μπορεί να έχει ταυτόχρονα πολλές ανοικτές συνδέσεις και να τις εξυπηρετεί πχ με νήματα, αλλά αυτό είναι εσωτερική υπόθεση της διεργασίας.
- Το λειτουργικό σύστημα, το TCP/IP, και πιθανώς το ενδιάμεσο λογισμικό, είναι υπεύθυνα για την ορθή διαχείριση της επικοινωνίας των διαφορετικών διεργασιών που εκτελούνται στον ίδιο υπολογιστή.

Δικτυακή επικοινωνία και μεταβίβαση μηνυμάτων

Χώρος διευθύνσεων διεργασίας A

Χώρος διευθύνσεων διεργασίας B



Στο κατανεμημένο υπολογισμό μας ενδιαφέρει η Λογική επικοινωνία (μηνύματα και πρωτόκολλο) και η Διεπαφή για να υλοποιηθεί η Φυσική επικοινωνία.

Επικοινωνία και ταυτοχρονισμός 1

- Ο κατανεμημένος υπολογισμός είναι ταυτόχρονος: κάθε διεργασία / νήμα έχει τη δική του ροή εκτέλεσης.
- Δεν υπάρχουν μοιραζόμενες μεταβλητές. Η κατάσταση κάθε διεργασίας / νήματος είναι ιδιωτική.
- Χωρίς επικοινωνία η κατάσταση του κατανεμημένου υπολογισμού είναι ένα απλό άθροισμα ανεξαρτήτων επιμέρους καταστάσεων.
- Ακριβώς όπως και νήματα ή διεργασίες χωρίς μοιραζόμενες μεταβλητές.

Επικοινωνία και ταυτοχρονισμός 2

- Η επικοινωνία επιτρέπει την αλληλεπίδραση των καταστάσεων των επιμέρους διεργασιών / νημάτων μέσω

(α) της ανταλλαγής δεδομένων με μεταβίβαση μηνυμάτων

(β) του συγχρονισμού στην παραλαβή (κυρίως) των μηνυμάτων.

- Ορίζουμε δύο ατομικές ενέργειες επικοινωνίας:

`send()`: αποστολή μηνύματος.

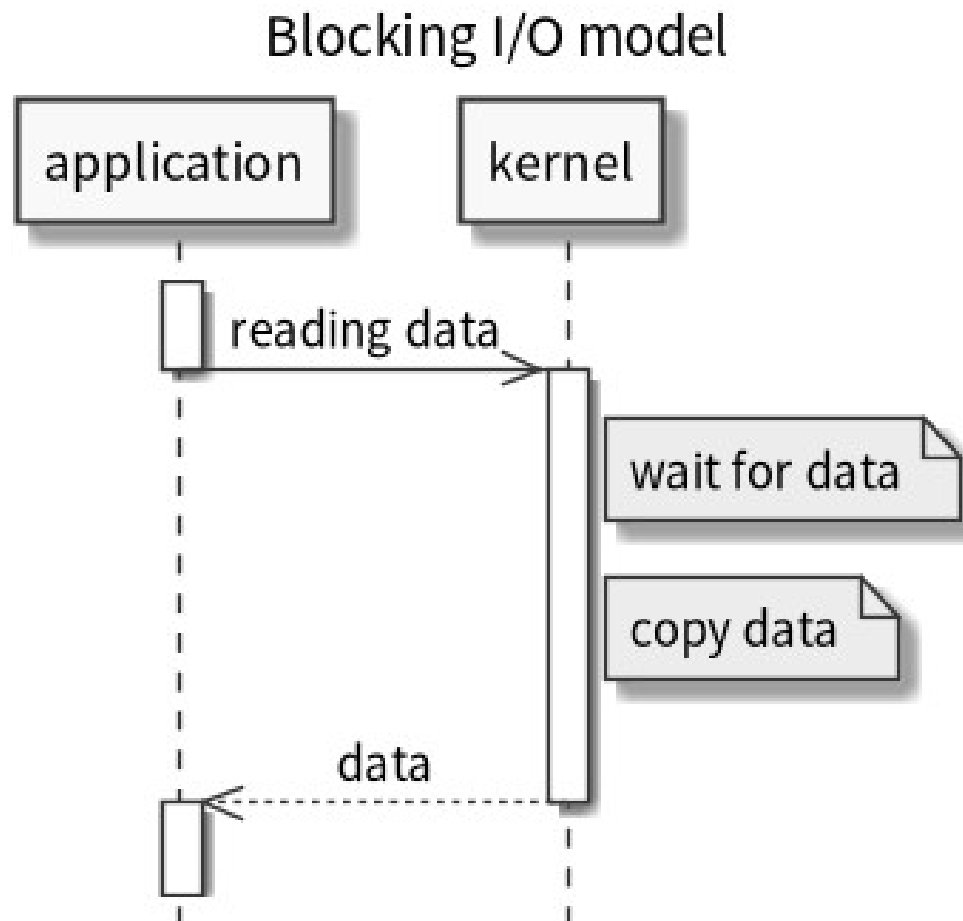
`recv()`: παραλαβή μηνύματος.

Επικοινωνία: συγχρονισμός στον παραλήπτη

- Από την πλευρά της αποστολής (συνήθως):
Μη-ανασταλτική (non-blocking) αποστολή : ο αποστολέας απλά περιμένει την επιστροφή της κλήσης συστήματος `send()` από το λειτουργικό σύστημα και συνεχίζει.
- Από την πλευρά της παραλαβής υπάρχουν διαφορετικές εναλλακτικές:
- Ανασταλτική (blocking)
- Μη-ανασταλτική (non-blocking)
- Με πολυπλεξία (multiplexed)
- Οδηγούμενη από σήματα (signal – driven)
- Ασύγχρονη (asynchronous)

Ανασταλτική παραλαβή (blocking I/O)

Βλ. <https://github.com/aliakh/demo-sockets-io-nio-nio2>



Ο παραλήπτης αναστέλλεται στο `recv()`. Είναι η πιο συνηθισμένη μέθοδος παραλαβής, αυτή που θα μελετήσουμε.



Ο παραλήπτης ρωτά κατά διαστήματα μέχρι το `recv()` να ολοκληρωθεί.

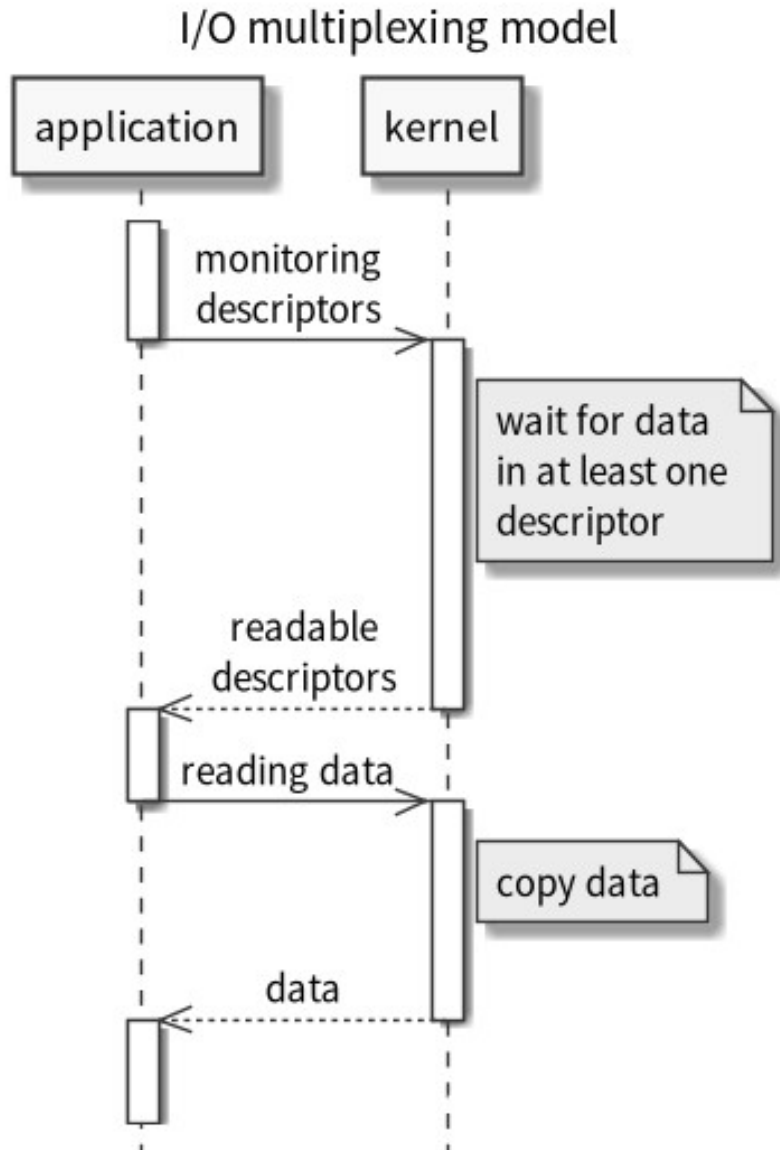
Ενεργός αναμονή!

Μπορεί όμως να εκτελεί και άλλες λειτουργίες ενδιάμεσα.

Επικάλυψη υπολογισμού και επικοινωνίας.

Αρκετά δύσκολη υλοποίηση.

Παραλαβή με πολυπλεξία (multiplexed I/O)

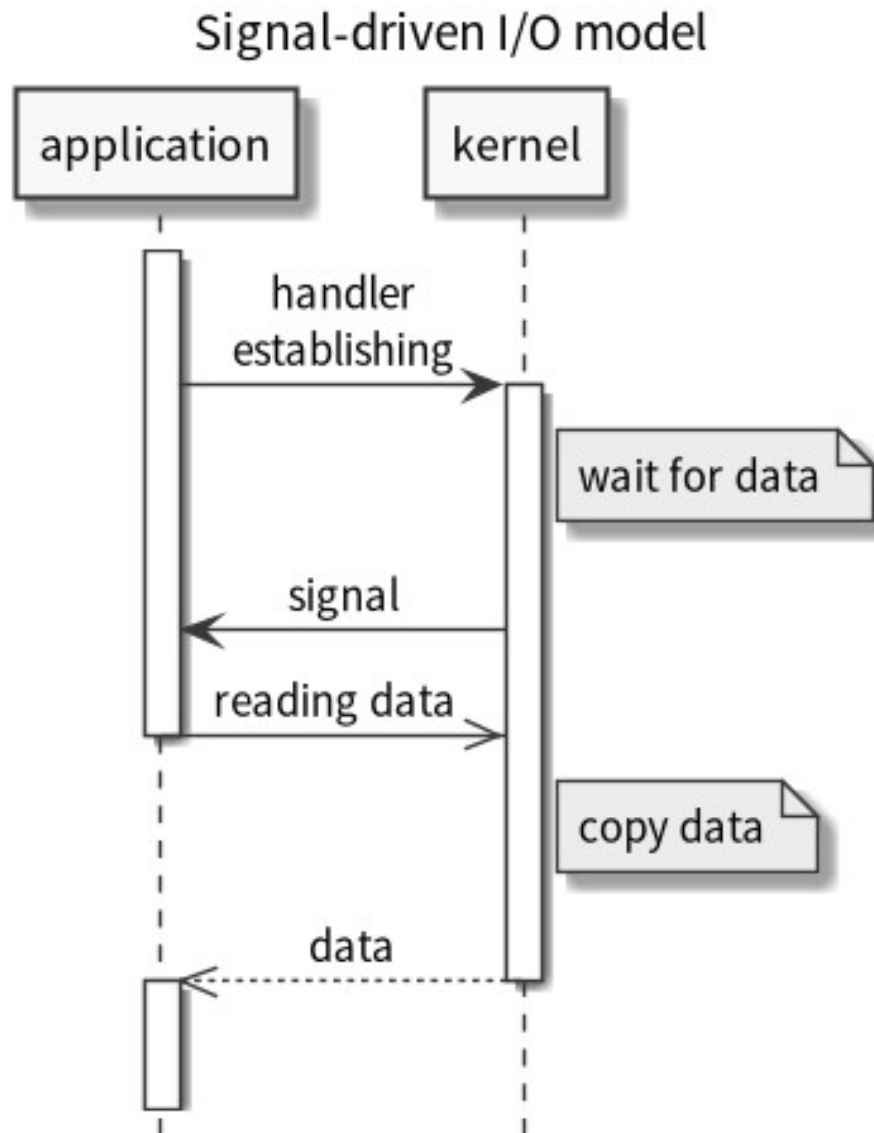


Χρήσιμη όταν ο παραλήπτης ελέγχει πολλές παραλαβές.

Επιλέγει αυτή που είναι διαθέσιμη και εκτελεί `recv()`.

Πολλαπλή ενεργός αναμονή και επιλογή παραλαβής.

Οδηγούμενη από σήματα (Signal-driven I/O)

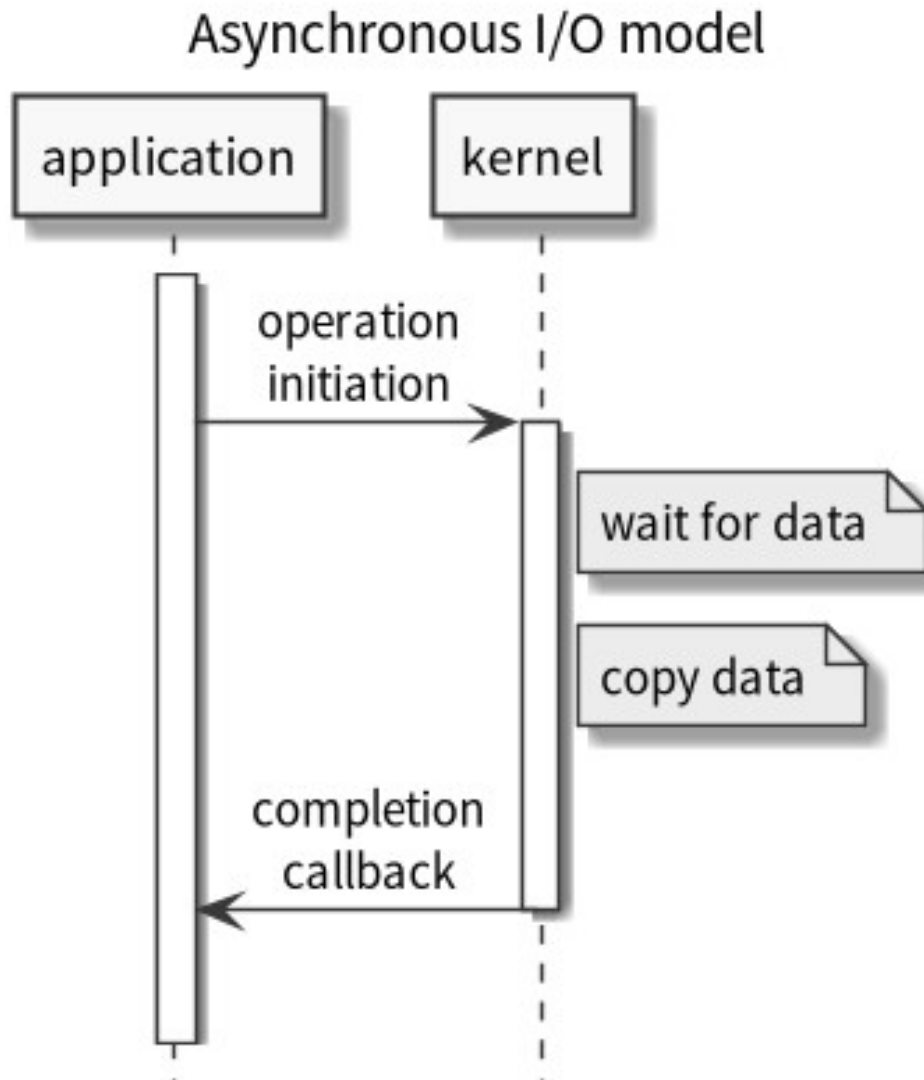


Ο παραλήπτης ξεκινά μια ρουτίνα διαχειριστή, που ενημερώνει με σήμα (interrupt) όταν το `recv()` μπορεί να εκτελεστεί.

Το `recv()` εκτελείται από τον παραλήπτη αλλά δεν υπάρχει αναστολή ή ενεργός αναμονή.

Μπορεί να υπάρχει επικάλυψη υπολογισμού και επικοινωνίας.

Ασύγχρονη παραλαβή (asynchronous I/O)



Ο παραλήπτης ξεκινά μια ρουτίνα διαχειριστή. Ο διαχειριστής αναστέλλεται στο `recv()` και εκτελεί τη παραλαβή.

Ο παραλήπτης ενημερώνεται με `callback()`, μια ασύγχρονη επιστροφή κλήσης του νήματος διαχειριστή.

Ατομικές ενέργειες επικοινωνίας

- Οι λειτουργίες `send()`, `recv()` είναι ατομικές ενέργειες κατ αναλογία της εγγραφής και ανάγνωσης σε μια θέση μνήμης.
- $x = y$
Load `y` ~ `send(y)`
Store `x` ~ `recv(x)`
- Επομένως ένα ζεύγος `send()` - `recv()` είναι λογικά ισοδύναμο με μια 'απομακρυσμένη' (ή εκτεταμένη) ανάθεση τιμής σε μεταβλητή.

$\text{send}() \rightarrow \text{recv}()$: ασθενής συνέπεια

- Ένα ζεύγος ατομικών ενεργειών $\text{send}()$ - $\text{recv}()$ ορίζει έναν κανόνα ασθενούς συνέπειας.
- Έστω δύο διεργασίες A, B. Η διεργασία A εκτελεί $\text{Send}()$ και η διεργασία B $\text{Recv}()$.
- Οι ακολουθίες εκτέλεσης των δύο διεργασιών έχουν τη παρακάτω επιπλέον εξάρτηση

... $\tau\alpha_1 \rightarrow \tau\alpha_2 \rightarrow \tau\alpha_3 \rightarrow \text{recv}() \rightarrow \tau\alpha_4 \rightarrow \tau\alpha_5 \rightarrow \tau\alpha_6 \dots$



... $\tau\beta_1 \rightarrow \tau\beta_2 \rightarrow \text{send}() \rightarrow \tau\beta_3 \rightarrow \tau\beta_4 \rightarrow \tau\beta_5 \rightarrow \tau\beta_6 \dots$

- Δηλαδή $\tau\beta_2 \rightarrow \text{send}() \rightarrow \text{recv}() \rightarrow \tau\alpha_4$

Επικοινωνία με ανασταλτική παραλαβή: Μηχανισμός ταυτοχρονισμού

- Επομένως το ζεύγος `send()` - `recv()` λειτουργεί ως μηχανισμός ταυτοχρονισμού μεταξύ διεργασιών.
- Ο μηχανισμός δεν περιλαμβάνει ενημέρωση μοιραζόμενων μεταβλητών, γιατί τέτοιες δεν υπάρχουν, αλλά περιλαμβάνει την ενημέρωση απομακρυσμένων μεταβλητών, αλλά με έλεγχο από τον παραλήπτη ('απομακρυσμένη' ανάθεση).
- Για τη κατάσταση του κατανεμημένου υπολογισμού μπορούμε ισχύει η ασθενής συνέπεια.

Αίτημα -απάντηση (request – reply)

- Ο αποστολέας περιμένει να παραλάβει από τον παραλήπτη ένα μήνυμα – απάντηση στο μήνυμα – αίτημα που έστειλε.
- Το μήνυμα – απάντηση μπορεί να είναι και απλή επιβεβαίωση παραλαβής ή εκτέλεσης.
- Το request-reply μπορεί να εκφραστεί ως δύο ζεύγη μη-ανασταλτικής αποστολής και ανασταλτικής παραλαβής που εκτελούνται από δύο διεργασίες:

A: send(x); recv(y)

B: recv(x); send(y)

Επικοινωνία: αναλογία με το συγχρονισμό σε συνθήκες 1

- Απλή σηματοδοσία

A: signal() A: send()

B: wait() B: recv()

απλή αποστολή
σύγχρονη παραλαβή

- Πολλαπλή σηματοδοσία

A: wait(y); signal(x) A: send(x); recv(y)

B: wait(x); signal(y) B: recv(x); send(y)

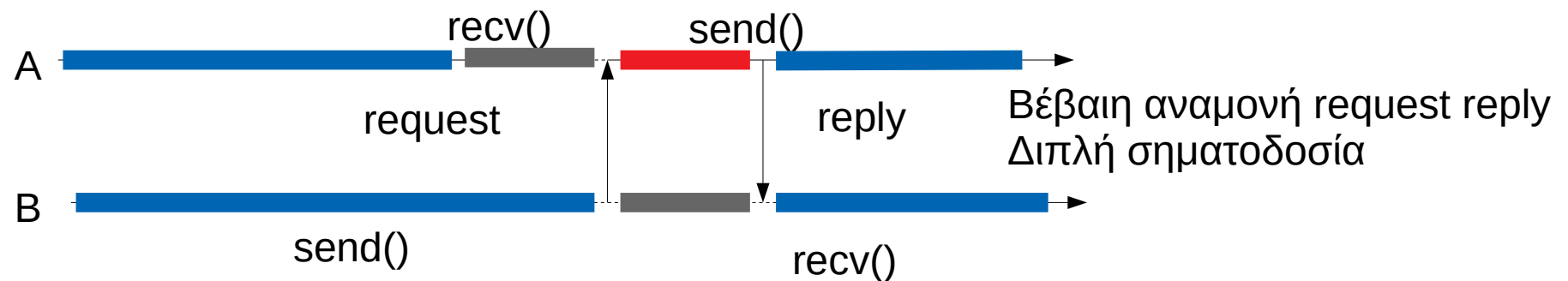
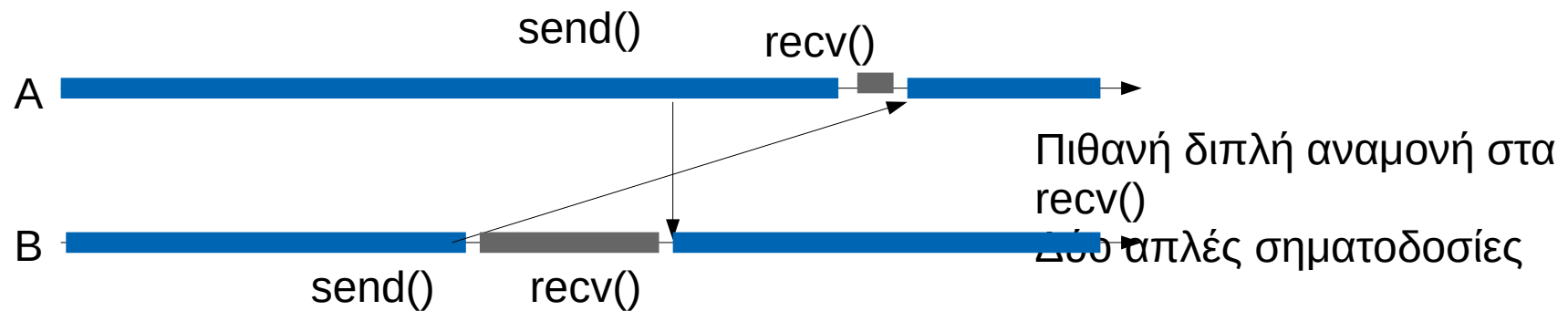
/

wait() ~ P(), signal ~ V()
ένα από τα x, y επιτρέπει
πρόσβαση

/

απλή αποστολή
σύγχρονη παραλαβή
ή αίτημα-απάντηση

Επικοινωνία: αναλογία με το συγχρονισμό σε συνθήκες 2



Επικοινωνία: αδιέξοδο και λιμοκτονία 1

- Λιμοκτονία

Σύγχρονη παραλαβή χωρίς αντίστοιχη αποστολή

A: recn() B: ...

- Αδιέξοδο

Wait-for-cylce

A: recn(y); send(x) B: recn(x); send(y)

Επικοινωνία: αδιέξοδο και λιμοκτονία 2

recv() send()

A —————>

B —————>

Αναστολή στο recv()
Δεν υπάρχει send()

recv() send()

A —————>

B —————>

Αδιέξοδο
Αναστολή στα recv()
Τα send() δεν θα εκτελεστούν

recv() send()

Επικοινωνία:

αναλογία με αμοιβαίο αποκλεισμό 1

- Ταυτοχρονισμός = απροσδιόριστη σειρά εκτέλεσης
- Ταυτοχρονισμός στην επικοινωνία

Αυθαίρετη (μη – αιτιοκρατική) σειριοποίηση
εξερχόμενων ή εισερχόμενων μηνυμάτων λόγω:

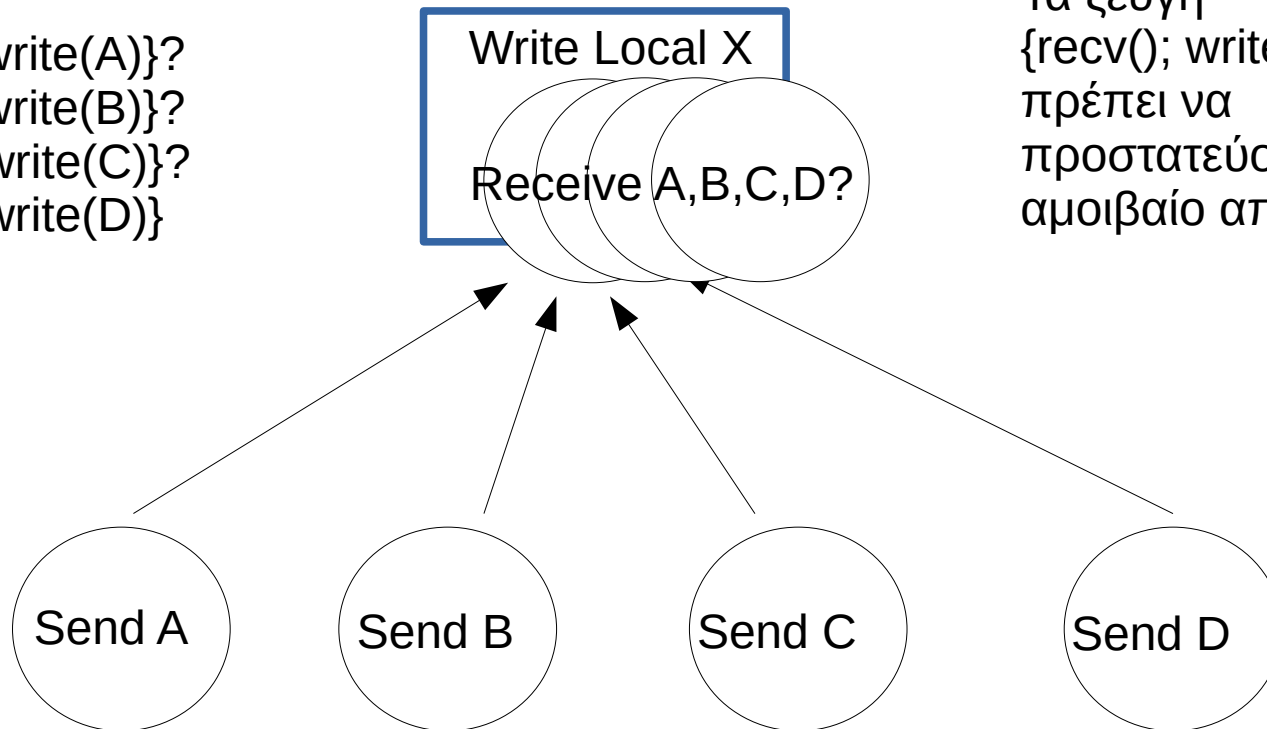
(α) Διεπαφής δικτυακής επικοινωνίας

(β) Κλήσεων συστήματος για `send()`, `recv()`

- Αναλογία με την πρόσβαση διεργασιών / νημάτων
στη μοιραζόμενη μνήμη.

Επικοινωνία: αναλογία με αμοιβαίο αποκλεισμό 2

{recv(A); write(A)}?
{recv(B); write(B)}?
{recv(C); write(C)}?
{recv(D); write(D)}



Τα ζεύγη
{recv(); write()}
πρέπει να
προστατεύονται με
αμοιβαίο αποκλεισμό

Επικοινωνία: αναλογία με αμοιβαίο αποκλεισμό 3

Τυπικές λύσεις

- Παραλαβή με πολυπλεξία
- Ασύγχρονη παραλαβή
- Πολλαπλές διεργασίες
- Πολλαπλά νήματα

Παραλαβή με πολυπλεξία, βρόχος συμβάντων

- Η επικοινωνία, δηλαδή το {recv(), write()}, μπορεί να αντιμετωπιστεί ως συμβάν (event).
- Το πρόγραμμα δομείται με βάση συμβάντα, όπως τοπική ή δικτυακή επικοινωνία με άλλα προγράμματα, επικοινωνία με το δίσκο, αλληλεπίδραση με τον χρήστη, κλπ.
- Ένας βρόχος (βρόχος συμβάντων, event loop) διατρέχει συνεχώς όλες τις πιθανές πηγές συμβάντων και επιλέγει και εξυπηρετεί κάθε φορά ένα συμβάν.

Πολλαπλές διεργασίες ή πολλαπλά νήματα

- Κάθε επικοινωνία μπορεί να εξυπηρετείται από μια διεργασία (πολύ δαπανηρό). Πιο συνηθισμένο: κάθε επικοινωνία εξυπηρετείται από ένα νήμα σε μια πολυνηματική διεργασία.
- **Νήμα ανά αίτημα** (Thread per request): όταν κάθε {send()-recv()} αποτελεί ξεχωριστή επικοινωνία (HTTP 1.x): πολύ δαπανηρό και ξεπερασμένο (βλ. HTTP 2.x, 3x)
- Συνήθως η επικοινωνία οργανώνεται **νήμα ανά σύνδεση** (Thread per connection)
- **Υβριδικά νήματα**: οι επικοινωνίες συνήθως έχουν μεγάλες καθυστερήσεις, επομένως πολλά υβριδικά νήματα μπορεί να εξυπηρετούνται από ένα νήμα επιπέδου λειτουργικού συστήματος.

Ασύγχρονη παραλαβή και εξυπηρέτηση

- Κάθε επικοινωνία εξυπηρετείται από μια ρουτίνα διαχείρισης που εκτελεί το αντίστοιχο {recv(), write()} και επιστρέφει ασύγχρονα (callback).
- Το κύριο πρόγραμμα δεν μπαίνει σε αναστολή εκτέλεσης (αναμονή για return της κάθε ρουτίνας) – ασύγχρονες κλήσεις.
- Συνεχίζει την εκτέλεσή του και οι ρουτίνες διαχείρισης επιστρέφουν το αποτέλεσμα τους σε ειδικές δομές.
- Χρήσιμη όταν η επικοινωνία πχ μέσω διαδικτύου παρουσιάζει μεγάλη καθυστέρηση, σε σχέση με την ταχύτητα της τοπικής επεξεργασίας.



Κατανεμημένος προγραμματισμός: Προγραμματισμός μεταβίβασης μηνυμάτων

Επέκταση του ακολουθιακού προγραμματισμού με ενσωματωμένη βιβλιοθήκη ή ανεξάρτητο API.

- Διαχείριση δικτυακής επικοινωνίας. Δημιουργία, τερματισμός, χαρακτηριστικά σύνδεσης.
- Κανόνες και μηχανισμοί μεταβίβασης μηνυμάτων.
- Διαχείριση διεργασιών / νημάτων (ή συνεργασία με ανάλογη βιβλιοθήκη ταυτόχρονου υπολογισμού).

Διεπαφές 1

- Berkeley Sockets: καθολικά αποδεκτή διεπαφή δικτυακής επικοινωνίας μέσω TCP (και UDP).
- Το απαιτούμενο ενδιάμεσο λογισμικό είναι ήδη εγκατεστημένο στον υπολογιστή.
- Τα λειτουργικά συστήματα και οι περισσότερες γλώσσες προγραμματισμού διαθέτουν τις απαραίτητες βιβλιοθήκες.
- Είναι πρότυπο χαμηλού επιπέδου, αποδοτικό και ανοικτό. Επιτρέπει την δικτυακή επικοινωνία μεταξύ εφαρμογών που εκτελούνται σε διαφορετικά υπολογιστικά συστήματα και είναι γραμμένες σε διαφορετικές γλώσσες.

Διεπαφές 2

Υπάρχουν πολλά άλλα πρότυπα:

- Πρόσθετες βιβλιοθήκες ανεξάρτητες από γλώσσες ή ενσωματωμένες σε γλώσσες προγραμματισμού.
- Καλύπτουν δικτυακή και τοπική επικοινωνία ή καλύπτουν μόνο τοπική επικοινωνία.
- Παρουσιάζουν γενικές ομοιότητες ή είναι εντελώς πρωτότυπα.

Το (δεύτερο) πιο διαδεδομένο πρότυπο:

- Κλήση απομακρυσμένης διαδικασίας
Remote procedure call (RPC).

Τοπική επικοινωνία και μεταβίβαση μηνυμάτων 1

- Επέκταση του μοντέλου δικτυακής επικοινωνίας στη τοπική επικοινωνία (σε ένα υπολογιστικό σύστημα).
- Οι διεργασίες εκτελούνται σε μοιραζόμενη μνήμη αλλά η γλώσσα και το σύστημα χρόνου εκτέλεσης επιβάλλουν μόνο (ή επιτρέπουν και) επικοινωνία με μεταβίβαση μηνυμάτων.
- Η μεταβίβαση μηνυμάτων εκτελείται μέσω του λειτουργικού συστήματος ή/και του συστήματος χρόνου εκτέλεσης της γλώσσας ή της εικονικής μηχανής.
- Παρόμοια λύση για την επικοινωνία μεταξύ νημάτων εντός μιας διεργασίας.

Τοπική επικοινωνία και μεταβίβαση μηνυμάτων 2

- Στη τοπική επικοινωνία η μεταβίβαση μηνυμάτων υλοποιείται συνήθως με μονόδρομες ή αμφίδρομες ουρές μηνυμάτων.
- Η εξυπηρέτηση πολλών διεργασιών ή νημάτων είναι επέκταση του μοντέλου Παραγωγός Καταναλωτής.

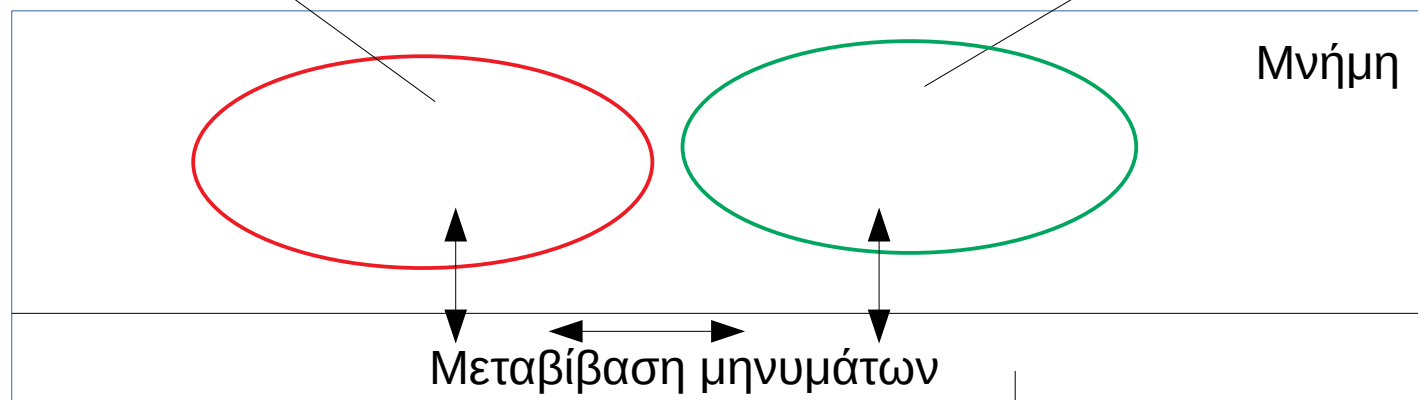
Υλοποίηση με:

- Βιβλιοθήκες του λειτουργικού συστήματος
- Βιβλιοθήκες της γλώσσας προγραμματισμού
- Τύπους και Δομές δεδομένων που είναι ενσωματωμένα στη γλώσσα προγραμματισμού

Τοπική επικοινωνία και μεταβίβαση μηνυμάτων 3

Χώρος διευθύνσεων διεργασίας A

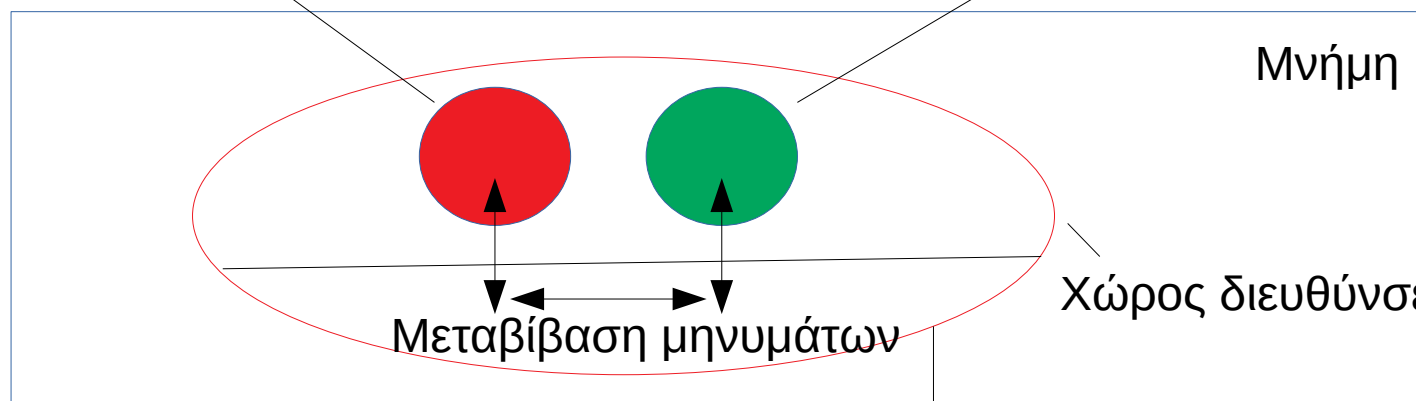
Χώρος διευθύνσεων διεργασίας B



Λειτουργικό σύστημα / Σύστημα χρόνου εκτέλεσης / Εικονική μηχανή

Νήμα A

Νήμα B



Σύστημα χρόνου εκτέλεσης / Εικονική μηχανή



Γιατί μεταβίβαση μηνυμάτων και όχι μοιραζόμενες μεταβλητές σε ένα σύστημα;

- Σχεδιαστική επιλογή:

η λογική του μη-διαμοιρασμού (share nothing) βοηθάει στην ανάπτυξη κώδικα που μπορεί ευκολότερα να ελεγχθεί, να συντηρηθεί και να κλιμακωθεί.

- Ομοιομορφία κώδικα:

σχεδόν (ή ακριβώς) ο ίδιος κώδικας για συστήματα μοιραζόμενης και κατανεμημένης μνήμης.

- Αντιμετώπιση προβλημάτων αναστολής:

ευκολότερη προσαρμογή σε λογικές μη-αναστολής εκτέλεσης κατά την αναμονή αποτελέσματος από τοπική ή απομακρυσμένη κλήση.